

Predictive Arbitration for Stateful Distributed Pipelines using Sequence Models

Robert Guy

Abstract

Modern distributed pipelines process heterogeneous streams of updates whose scheduling decisions impact not only throughput, but also tail latency [1], fairness, and the freshness of shared state [6]. Prior work has shown that adaptive arbitration policies based on runtime signals can improve these trade-offs, but such approaches remain fundamentally reactive and often fail under dynamic or non-stationary workloads.

In this work, we introduce predictive arbitration for stateful distributed pipelines using sequence models. Rather than selecting actions based solely on current system state, the proposed scheduler uses recent pipeline history to forecast the future cost of candidate decisions and selects the action minimizing predicted downstream impact. We formulate arbitration as a predictive control problem in which a sequence model estimates latency and freshness consequences over a finite horizon.

We evaluate the approach in a simulation environment with heterogeneous update streams and non-stationary workloads, comparing against FIFO, static-priority scheduling, and adaptive priority heuristics. Results show that predictive arbitration improves average-case performance and adapts more effectively to workload shifts, but remains sensitive to rare, high-cost events that dominate tail latency.

These findings highlight a fundamental limitation of learned scheduling approaches: while forecasting improves typical system behavior, accurately modeling and controlling tail behavior remains challenging. This motivates future work on incorporating uncertainty and tail-aware objectives into predictive schedulers.

1 Introduction

Stateful distributed pipelines are increasingly used to support real-time data processing across domains such as telemetry analysis, streaming analytics, autonomous systems, and distributed control. In these environments, streams of heterogeneous updates propagate through multi-stage processing pipelines that maintain evolving system state. The order in which updates are processed directly shapes system behavior, influencing not only throughput but also state freshness, tail latency, and responsiveness.

Traditional scheduling policies used in such systems are typically reactive. First-in-first-out queues, static priorities, and fair-share policies select work based on instantaneous queue conditions or fixed priority assignments. While such policies can perform adequately under stable workloads, they often struggle in dynamic environments where traffic patterns fluctuate, bursts occur, or workloads exhibit temporal structure. Reactive schedulers are inherently limited: they optimize decisions based only on the current state of the system, without anticipating how those decisions will influence future pipeline behavior.

Recent work, including our prior work on adaptive priority arbitration [11], has explored mechanisms that incorporate runtime signals such as queue depth, latency, and freshness to

dynamically adjust scheduling decisions. While these approaches improve responsiveness to changing conditions, they remain fundamentally reactive and rely on aggregate statistics that obscure rare but high-impact events, leading to persistent tail latency under dynamic workloads. Moreover, these approaches rely on aggregate statistics (e.g., mean latency or queue depth) that may fail to capture rare but high-impact events, leading to persistent tail latency and delayed recovery under dynamic workloads.

This paper investigates whether arbitration decisions can instead be guided by forecasting future system behavior. If a system can predict the downstream latency and freshness consequences of candidate actions, it can proactively select decisions that mitigate emerging congestion and state staleness before they manifest.

To study this question, we formulate arbitration as a **predictive control problem**. At each scheduling step, the system observes recent pipeline telemetry including queue depths, arrival patterns, latency indicators, freshness lag, and prior scheduling actions and uses a sequence model to forecast the downstream cost of potential scheduling choices, similar in spirit to prior work on learning-based resource management [3, 4]. The scheduler then selects the action with the lowest predicted future cost. This formulation reframes arbitration as a predictive control problem, transforming scheduling from a reactive mechanism into one guided by forecasts of future system cost.

We instantiate this framework using sequence models that learn temporal patterns in pipeline behavior. By modeling the recent evolution of the system state, the predictor estimates how candidate scheduling actions will influence future latency and freshness metrics over a finite horizon. This design allows arbitration decisions to incorporate both current system conditions and anticipated future dynamics. Our goal is to determine whether forecasting-driven arbitration can improve the trade-off between state freshness and tail latency under dynamic conditions, and to characterize the limitations of learned scheduling in capturing rare, high-impact events that dominate tail latency.

We evaluate the proposed approach using a simulation-based model of a multi-stage distributed pipeline with heterogeneous update streams and non-stationary workloads. The predictive scheduler is compared against several baseline policies, including FIFO, static-priority scheduling, and a hand-designed adaptive arbitration strategy. Our goal is to determine whether forecasting-driven arbitration can improve the trade-off between state freshness and tail latency under dynamic conditions.

The contributions of this work are threefold:

- We formulate arbitration in stateful distributed pipelines as a predictive control problem, where decisions are guided by forecasts of future system cost.
- We introduce a sequence-model-based arbitration framework that learns temporal workload dynamics from pipeline telemetry and uses these predictions to guide scheduling decisions.
- We provide an empirical evaluation under dynamic, non-stationary workloads, demonstrating consistent improvements in average-case performance while revealing fundamental limitations in controlling tail latency.

Together, these results suggest that predictive models of pipeline dynamics provide a strong foundation for proactive scheduling, but are insufficient on their own to robustly control tail latency in dynamic, stateful systems. This highlights the need for tail-aware or uncertainty-aware scheduling mechanisms that explicitly account for rare but high-impact events [1].

2 System Model

Distributed processing pipelines consist of multiple stages that transform or aggregate streams of updates to produce a derived system state. Updates from heterogeneous sources propagate through sequences of processing stages connected by intermediate queues. Arbitration arises whenever multiple pending updates compete for processing at a stage with limited service capacity.

We model the system as a pipeline of S processing stages connected by finite queues. Each stage processes jobs drawn from multiple update streams. At each scheduling step, the system selects an action $a \in \mathcal{A}$, where each action corresponds to choosing the next job from a particular stream or queue class.

At each scheduling step t , the system observes the current pipeline state x_t , which summarizes runtime telemetry describing system conditions. This state may include features such as queue depths, recent arrival counts, latency indicators, freshness lag, and recent scheduling actions. To capture temporal dynamics, decisions are based on a history of recent observations:

$$h_t = (x_{t-k+1}, x_{t-k+2}, \dots, x_t)$$

where k denotes the length of the observation history used by the predictive model.

Traditional reactive schedulers select actions using only the current state x_t , without explicitly modeling how decisions influence future system behavior. In contrast, we consider predictive arbitration strategies that estimate the downstream impact of candidate actions over a finite horizon.

For each candidate action a , we define a future cost over a finite horizon H that captures system objectives including latency, freshness, and fairness. The cost is modeled as a weighted combination of these components [8]:

$$J^{(H)}(a) = \alpha \cdot L^{(H)}(a) + \beta \cdot F^{(H)}(a) + \gamma \cdot S^{(H)}(a)$$

where $L^{(H)}(a)$ represents latency-related cost, $F^{(H)}(a)$ captures freshness lag, and $S^{(H)}(a)$ represents penalties for imbalance or starvation across streams. The coefficients α , β , γ control the relative importance of these objectives. In this work, we fix $\alpha = 1.0$, $\beta = 0.2$, and $\gamma = 4.0$, reflecting a deliberate design choice in the formulation of the objective.

These weights reflect system level priorities and remain fixed during training and evaluation. As a result, the predictive model learns to estimate future cost under a predefined objective, rather than implicitly defining the objective itself.

Because the true future cost cannot be known at decision time, the scheduler relies on a learned predictor f_θ to estimate the expected cost of each candidate action given recent system history:

$$\hat{J}(h_t, a) = f_\theta(h_t, a)$$

The scheduler selects the action that minimizes the predicted cost:

$$a_t^* = \underset{a \in \mathcal{A}}{\operatorname{argmin}} \hat{J}(h_t, a)$$

This formulation depends on the accuracy of the predictive model. In practice, prediction errors – particularly under non-stationary workloads [9] or rare events can lead to suboptimal scheduling decisions, especially when tail behavior dominates system performance.

In this work, we instantiate f_θ using sequence models that learn temporal patterns in pipeline telemetry [10]. By modeling the recent evolution of system state, the predictor estimates how

candidate scheduling decisions influence future latency and freshness dynamics. This enables arbitration decisions to incorporate both current system conditions and anticipated future behavior, while also exposing the limitations of learned prediction in dynamic environments.

3 Predictive Arbitration via Sequence Models

The system model in **Section 2** defines arbitration as selecting the action that minimizes predicted future system cost. We now describe how this prediction is realized using a sequence model over recent pipeline telemetry. The key insight is that effective arbitration requires reasoning over the temporal evolution of system state, rather than relying solely on instantaneous observations. In particular, queue dynamics, burst arrivals, and delayed congestion effects introduce temporal dependencies that are not captured by reactive scheduling policies.

3.1 Observation Sequence

At each scheduling step t , the system records a feature vector x_t summarizing the current state of the pipeline. While the exact feature set may vary by implementation, we use the following features:

- queue depth for each stream or queue class,
- recent arrival counts over a fixed observation window,
- estimated freshness lag of the system state,
- recent latency indicators or waiting-time summaries, and
- the most recent arbitration action.

Rather than making decisions from a single observation, the scheduler operates over a history window of length k :

$$h_t = (x_{t-k+1}, x_{t-k+2}, \dots, x_t)$$

This sequence captures temporal patterns such as burst arrivals, backlog accumulation, and delayed recovery following congestion. Many scheduling pathologies arise from these evolving dynamics rather than from a single instantaneous snapshot, making historical context essential for effective arbitration.

3.2 Sequence Model Architecture

We instantiate the predictive model f_θ using a recurrent sequence model that encodes recent pipeline telemetry. In particular, we use a long short-term memory (LSTM) network [10] to model temporal dependencies in the observation history h_t . The LSTM processes the sequence and produces a latent representation that summarizes recent system dynamics, including queue evolution and workload fluctuations.

Given the encoded history and a candidate arbitration action $a \in \mathcal{A}$, the model predicts the expected downstream cost of selecting that action. Importantly, the model does not directly output a scheduling decision. Instead, it estimates the future consequence of each possible action, and the scheduler selects the action with minimum predicted cost. This design preserves interpretability and maintains a clear separation between prediction and decision-making.

3.3 Action-Conditioned Cost Prediction

For each candidate action a , the predictive model computes:

$$\hat{J}(h_t, a) = f_\theta(h_t, a)$$

where $\hat{J}(h_t, a)$ represents the predicted cost incurred over the next H decision steps if action a is taken at time t . In practice, this is implemented by concatenating an embedding of the candidate action with the latent representation of the telemetry history and passing the result through a prediction head.

This action-conditioned formulation allows the scheduler to evaluate all candidate decisions at each time step and select the one with minimum predicted downstream cost:

$$a_t^* = \underset{a \in \mathcal{A}}{\operatorname{argmin}} \hat{J}(h_t, a)$$

Because the model evaluates actions independently, it can be interpreted as learning a cost surface over actions conditioned on recent system behavior.

3.4 Training Objective

The predictive model is trained using supervised regression in a simulation environment. For each decision point, we record the recent telemetry sequence h_t , enumerate candidate actions, and compute the realized future cost $J^{(H)}(a)$ over a finite horizon H .

The model is trained to minimize the mean squared error between predicted and realized cost. The realized cost $J^{(H)}(a)$ is computed using the weighted objective defined in **Section 2**, ensuring that the learned model aligns with the system’s operational priorities.

$$\mathcal{L}(\theta) = \sum_t \sum_{a \in \mathcal{A}} (f_\theta(h_t, a) - J^{(H)}(a))^2$$

This objective encourages the model to learn temporal patterns that are predictive of downstream latency growth, freshness degradation, and imbalance across streams. However, because the model is trained on finite-horizon outcomes and observed trajectories, it may underrepresent rare but high-impact events that disproportionately contribute to tail latency.

3.5 Design Rationale

We use sequence models for two primary reasons. First, the arbitration problem is inherently temporal: queue buildup, burst amplification, and freshness degradation evolve over time rather than appearing instantaneously. Second, recent system history often contains structure that is not captured by simple hand-designed scoring rules. A sequence model can learn recurring workload patterns such as short bursts, sustained overload, or delayed recovery following congestion.

Compared to the hand-designed adaptive scheduler introduced in prior work [11], the sequence-model-based scheduler replaces manually tuned decision weights with a learned predictor of future arbitration cost. This preserves the structure of the arbitration problem while allowing the policy to adapt to temporal workload behavior that may be difficult to encode analytically.

At the same time, this approach introduces a dependency on prediction accuracy. In dynamic or non-stationary workloads, prediction errors – particularly for rare events can lead to suboptimal decisions and persistent challenges in controlling tail latency.

Furthermore, this approach differs from reinforcement learning-based schedulers, as it does not require policy optimization or exploration, but instead learns a predictive model of future cost that is used within a structured decision rule. This reduces training complexity while preserving the ability to incorporate domain-specific objectives.

4 Evaluation Methodology

We evaluate the proposed predictive arbitration framework using a simulation-based model of a multi-stage distributed pipeline. The goal of the evaluation is to determine whether forecast-driven scheduling decisions can improve system responsiveness and state quality relative to conventional reactive scheduling strategies under dynamic workloads.

In particular, we investigate whether sequence-model-based arbitration can improve tail latency, maintain fresher system state, and reduce scheduling instability when workloads exhibit temporal variation such as burst arrivals or sustained load shifts.

4.1 Simulation Environment

The evaluation uses a discrete-event simulation of a multi-stage distributed processing pipeline composed of multiple stages connected by finite queues. Each stage processes jobs drawn from heterogeneous update streams, which represent logically distinct classes of work with differing arrival rates, service costs, and importance to system state.

Jobs are generated by upstream producers and propagate through the pipeline until completion. Each stage has limited processing capacity and must arbitrate among competing jobs whenever multiple updates are pending. As a result, arbitration decisions directly influence queue growth, end-to-end latency, and the freshness of the derived system state.

We model three representative workload classes:

- **Control streams**, representing latency-sensitive updates that must be processed quickly to maintain responsive system behavior.
- **Burst streams**, which generate large volumes of updates over short time intervals.
- **Background streams**, representing steady but lower-priority processing tasks.

These workload classes approximate heterogeneous update patterns commonly observed in telemetry processing, streaming analytics, and distributed control systems.

4.2 Workload Dynamics

To evaluate adaptive behavior, workloads are generated with time-varying arrival rates. The simulation includes phases of stable traffic, sudden burst arrivals, and periods of sustained overload. These conditions create transient congestion events that require effective arbitration to prevent latency escalation and excessive state staleness.

Workload parameters such as arrival rate and burst duration are varied across experiments to evaluate the robustness of scheduling strategies under different levels of contention.

4.3 Baseline Scheduling Policies

We compare the predictive scheduler against several baseline policies commonly used in distributed systems:

- **FIFO scheduling**, which processes jobs strictly in arrival order.
- **Static-priority scheduling**, which assigns fixed priority levels to different streams.
- **Adaptive priority arbitration**, a hand-designed dynamic scheduling policy introduced in prior work.

These baselines represent a range of reactive strategies, providing a reference for evaluating the benefits of predictive arbitration.

4.4 Training Data Generation

Training data for the sequence model is generated by executing the simulation under a mixture of baseline scheduling policies and recording pipeline telemetry at each decision step. For each observed system history h_t , candidate actions are enumerated and the realized future cost $J^{(H)}(a)$ is computed over a fixed horizon H .

Each training example therefore consists of:

- a sequence of recent pipeline observations h_t ,
- a candidate arbitration action a , and
- the realized downstream cost $J^{(H)}(a)$.

This dataset allows the model to learn relationships between recent pipeline dynamics and the downstream consequences of scheduling decisions. However, because the data is generated from finite simulations and observed trajectories, rare but high-impact events may be underrepresented in the training distribution.

4.5 Evaluation Metrics

Scheduler performance is evaluated using several system-level metrics. In some experiments, latency components are reweighted to emphasize tail behavior, allowing us to assess the sensitivity of predictive arbitration to rare but high-impact events. In all experiments, the objective weights are fixed to $\alpha = 1.0$, $\beta = 0.2$, and $\gamma = 4.0$ as defined in **Section 2**.

- **Mean latency**, measuring the average time jobs spend in the pipeline.
- **Tail latency**, including P95 and P99 latency measurements.
- **Latency variance**, reflecting scheduling stability.
- **State freshness lag**, representing the delay between event generation and visible state updates.
- **Starvation indicators**, measuring imbalance across streams.

These metrics collectively capture the trade-off between responsiveness, fairness, and state quality that arbitration policies must manage. In particular, tail latency metrics are emphasized, as rare but extreme delays often dominate perceived system performance in dynamic workloads.

5 Experimental Results

We evaluate the proposed predictive arbitration framework across multiple random seeds and report averaged system-level metrics. Table 1 and Figures 1-2 summarize performance across FIFO, static-priority, adaptive-priority, and predictive scheduling policies.

Overall, predictive arbitration consistently improves average-case system performance, achieving the lowest mean latency and lowest mean freshness lag across all evaluated policies. However, these improvements do not uniformly extend to tail latency, revealing a key limitation of forecasting-based scheduling under rare but high-impact congestion events.

Compared to reactive baselines, predictive scheduling reduces average job completion time by approximately 14% relative to FIFO and 8% relative to static-priority scheduling. For example, mean latency decreases from approximately 10.1 seconds under FIFO to 8.7 seconds under predictive arbitration, while also improving state freshness. Predictive arbitration additionally achieves the lowest mean freshness lag, indicating improved responsiveness of the visible system state.

Table 1: Average performance across scheduling policies over three random seeds. Lower values indicate better performance for all latency and freshness metrics. Predictive arbitration achieves the best mean latency and mean freshness lag, but does not achieve the best tail latency.

Policy	Mean Lat.	P95	P99	Lat. Std.	Mean Fresh.	Max Fresh.
FIFO	10133.51	20035.84	20187.87	6795.89	5581.51	20223.85
Static Priority	9465.20	21396.37	22548.69	7357.01	3848.43	20222.32
Adaptive Priority	10076.62	20039.30	20270.78	6801.81	5404.97	20222.32
Predictive	8713.28	20256.38	20903.00	7036.47	3632.72	20221.72

However, improvements in average-case performance do not uniformly translate to reductions in tail latency. While predictive scheduling performs competitively in P95 latency, it does not consistently outperform reactive baselines in P99 latency. In some cases, adaptive-priority scheduling achieves slightly lower tail latency, suggesting that rare but high-impact congestion events remain difficult to anticipate using learned forecasting alone.

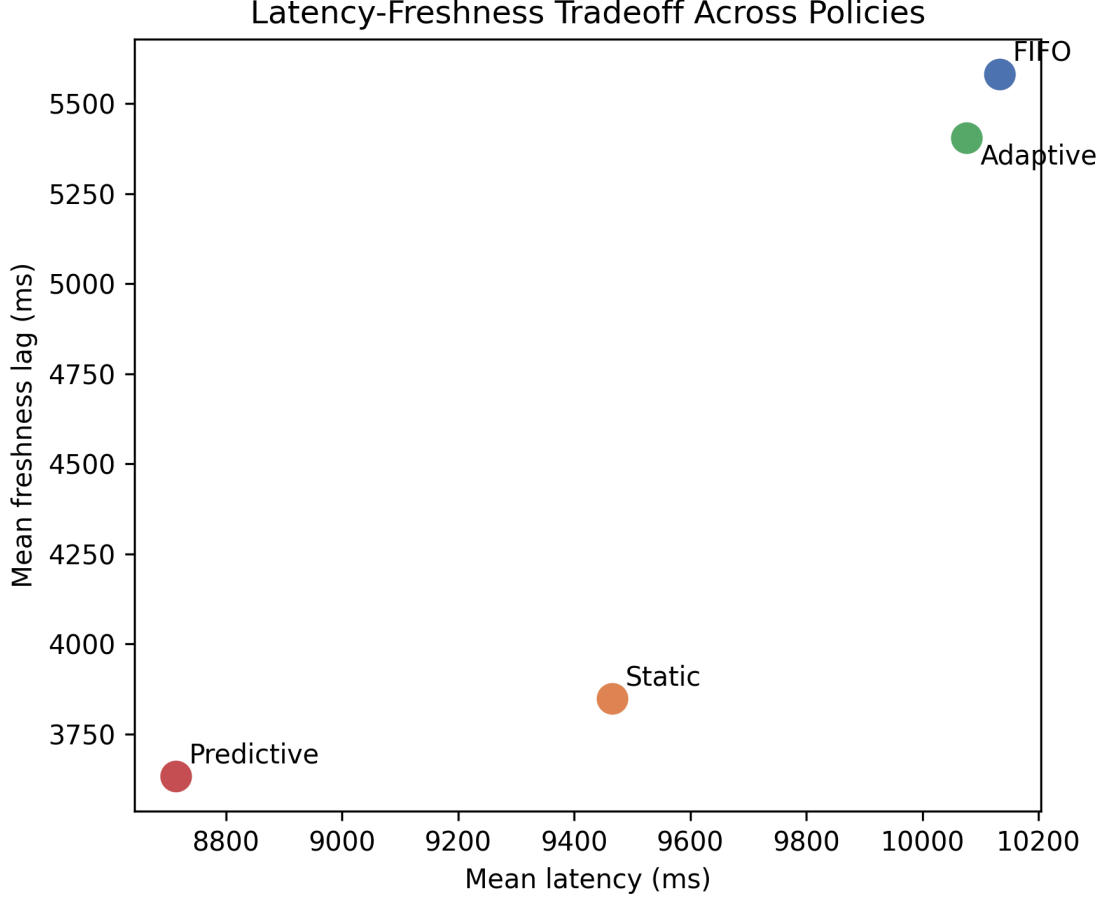


Figure 1: Latency–freshness tradeoff across scheduling policies. Predictive arbitration occupies the most favorable region of the tradeoff space, achieving both lower mean latency and lower mean freshness lag than the reactive baselines.

While this trade-off highlights the benefits of predictive arbitration, it also reveals an important limitation of forecasting-based approaches. Although the sequence model captures temporal workload patterns and enables proactive decision-making, rare events may be underrepresented in training data and therefore not fully mitigated by learned policies. As a result, predictive arbitration improves typical system behavior, but does not reliably mitigate extreme tail latency events.

Figure 1 illustrates the trade-off between mean latency and mean freshness lag across scheduling policies. Predictive arbitration occupies the most favorable region of the trade-off space, achieving both lower latency and improved state freshness relative to reactive baselines. However, while this trade-off highlights the benefits of predictive arbitration in typical operating conditions, it also reveals a key limitation. Although the sequence model captures temporal workload patterns and enables proactive decision-making, rare events may be underrepresented in training data and therefore not fully mitigated by learned policies. As a result, predictive arbitration improves typical system behavior, but does not reliably mitigate extreme tail latency events. Figure 2 further illustrates performance across key metrics. Predictive arbitration achieves the lowest mean latency and lowest mean freshness lag, but these improvements do not extend to P99 latency. This reinforces that average-case gains do not fully resolve extreme

delay events.

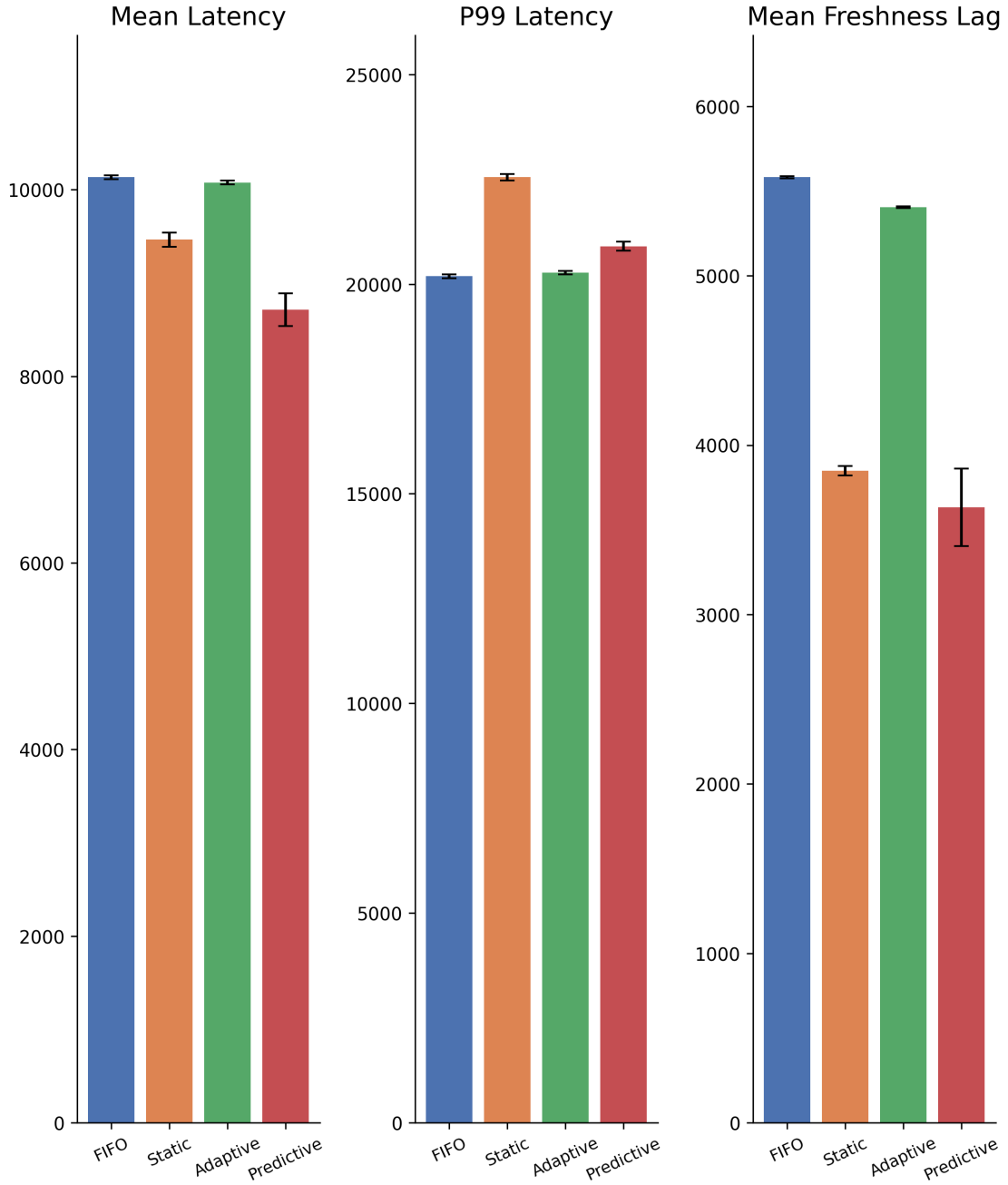


Figure 2: Average performance across scheduling policies. Predictive arbitration achieves the lowest mean latency and lowest mean freshness lag, but these improvements do not extend to P99 latency, indicating that average-case gains do not fully resolve extreme delay events.

Finally, the results in Table 1 indicate that predictive scheduling does not significantly reduce latency variance compared to reactive approaches. This suggests that forecasting alone is insufficient for robust tail control, and must be complemented by tail-aware or uncertainty-aware mechanisms, such as risk-sensitive objectives (e.g., CVaR) [5].

6 Conclusion

In this work, we presented a predictive arbitration framework for real-time scheduling that leverages sequence-based modeling to anticipate workload dynamics and guide scheduling decisions. Across multiple scheduling policies and randomized workloads, predictive arbitration consistently improves average-case system performance, achieving lower mean latency and reduced freshness lag compared to reactive baselines.

However, our results reveal a key limitation of forecasting-based approaches. While predictive scheduling captures temporal workload structure and enables proactive decision-making, it does not reliably improve tail latency or reduce latency variability. In particular, improvements in mean latency do not consistently extend to P99 latency, indicating that rare but high-impact congestion events remain difficult to anticipate and mitigate through prediction alone.

These findings suggest that predictive models alone are insufficient for robust control of extreme delay events. This highlights the need for scheduling frameworks that explicitly account for tail risk and uncertainty, rather than relying solely on expected-value predictions. Future work will explore integrating tail-aware optimization objectives, such as risk-sensitive or CVaR-based formulations, to more effectively manage worst-case latency while preserving the benefits of predictive scheduling.

Overall, this work establishes predictive arbitration as a promising direction for improving real-time system performance, while motivating hybrid approaches that combine forecasting with explicit tail-latency control mechanisms.

References

- [1] Jeffrey Dean and Luiz André Barroso, *The Tail at Scale*, Communications of the ACM, vol. 56, no. 2, pp. 74–80, 2013.
- [2] Leonard Kleinrock, *Queueing Systems, Volume 1: Theory*, Wiley, 1975.
- [3] Hongzi Mao, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula, *Resource Management with Deep Reinforcement Learning*, in *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets)*, pp. 50–56, 2016.
- [4] Azalia Mirhoseini, Hieu Pham, Quoc V. Le, Benoit Steiner, Rasmus Larsen, Yuichi Zhou, Naveen Kumar, Rupak Shrestha, Martin Tancik, Daphne Amodei, et al., *Device Placement Optimization with Reinforcement Learning*, in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pp. 2430–2439, 2017.
- [5] R. Tyrrell Rockafellar and Stanislav Uryasev, *Optimization of Conditional Value-at-Risk*, Journal of Risk, vol. 2, no. 3, pp. 21–41, 2000.
- [6] Sanjit Kaul, Roy Yates, and Marco Gruteser, *Real-Time Status: How Often Should One Update?*, in *Proceedings of the IEEE INFOCOM*, pp. 2731–2735, 2012.
- [7] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica, *Dominant Resource Fairness: Fair Allocation of Multiple Resource Types*, in *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pp. 323–336, 2011.
- [8] S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004.
- [9] G. Widmer and M. Kubat, "Learning in the Presence of Concept Drift and Hidden Contexts," *Machine Learning*, vol. 23, no. 1, pp. 69-101, 1996.
- [10] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [11] Robert Guy, *Adaptive Priority Arbitration for Stateful Distributed Pipelines*, Zenodo, 2026. Available: <https://zenodo.org/records/19026183>